Play Store: 4.5-Star App Predictor

Support Vector Machine (SVM) | Binary Classification

Student Name: _____

Roll Number: _____

Enrollment Number: _____

Division: _____

Machine Learning Project Report

Complete personal details and insert the exact evaluation scores printed by your Jupyter Notebook before submission.

1. Introduction

This project analyses Google Play Store applications using **googleplaystore.csv**. Its prediction task is to identify highly rated apps, with **top_app = 1** when Rating is at least 4.5 and 0 otherwise.

2. Problem statement

The aim is to explore which app characteristics are associated with high ratings and build a binary classifier. Candidate attributes include category, reviews, size, installs, free/paid type, price, content rating and genre.

3. Dataset overview

The dataset is from the Kaggle Google Play Store Apps collection. Confirm row and column counts after loading the CSV. The project guide specifies the file googleplaystore.csv.

4. Data preparation

• Convert Rating to numeric and create top_app from Rating ≥ 4.5.

• Remove duplicate app records.

• Clean Installs by removing plus signs and commas.

• Convert Size values such as '19M' into numeric megabytes and handle 'Varies with device'.

• Remove currency symbols from Price and convert it to numeric.

• Inspect the malformed shifted row where Rating is 19 and remove it.

• Exclude App and Rating from model inputs to avoid identifiers and target leakage.

5. Exploratory data analysis

• Plot the top_app class distribution to inspect imbalance.

• Use histograms for numeric columns such as Reviews, Installs and Price.

• Compare a numeric feature across target classes with a box plot.

• Create a correlation heatmap for numeric features.

• Write five insights directly supported by the generated charts.

6. Feature engineering and encoding

Create at least one meaningful feature from existing columns and explain it. One-hot encode categorical variables such as Category, Type, Content Rating and Genres. Use SelectKBest with f_classif to select approximately 15 useful features, fitting the selector only on training data in a leakage-safe workflow.

7. Model: Support Vector Machine

The assigned model is Support Vector Machine (SVM). SVM finds a decision boundary that separates classes. Since feature scales differ greatly, standardize features. Fit the scaler on training data only, then transform both training and test data.

8. Training and testing

Use an 80/20 train/test split with random_state=42 and stratify=y. This preserves the target class ratio in both sets. If the dataset is large, use a reproducible sample as specified in the guide.

9. Evaluation

Evaluate the test set with accuracy, precision, recall, F1-score, specificity and ROC-AUC. Display a confusion matrix and classification report. Enter the exact values printed by your notebook.

Evaluation results

Metric	Notebook result
Accuracy	_____
Precision	_____
Recall	_____
F1-score	_____
Specificity	_____
ROC-AUC	_____

10. Model comparison

Compare at least four SVM configurations, such as different kernels or values of C, and record each result. Use F1-score and ROC-AUC as the main metrics. Select the best setting based on the comparison table and explain the trade-off between false positives and false negatives.

11. Conclusion

This project combines app-store data cleaning, exploratory analysis, feature engineering and SVM classification. It estimates whether an app is likely to meet the 4.5-star threshold using the selected app attributes. Results depend on dataset quality and should be reported using the actual held-out test metrics.